

Intelligent Insurance Bundle Recommendation System

DataQuest Hackathon

1 Problem Definition

The objective is to predict the final `Purchased_Coverage_Bundle` selected by a customer among 10 possible classes (0–9). The evaluation metric is the Macro F1-Score, with additional penalties applied for model size and inference latency.

2 System Architecture Overview

The system is divided into three logical layers:

1. Feature Engineering Layer
2. Prediction Layer (Random Forest Classifier)
3. Business Optimization Layer (Expected Gain Maximization)

3 Feature Engineering

Feature engineering focuses on enhancing signal quality while maintaining model efficiency.

Examples of engineered features:

- Total Dependents:

$$\text{TotalDependents} = \text{Adult} + \text{Child} + \text{Infant}$$

- Income per Dependent:

$$\text{IncomePerDependent} = \frac{\text{EstimatedAnnualIncome}}{\text{TotalDependents} + 1}$$

- Claims Ratio:

$$\text{ClaimsRatio} = \frac{\text{PreviousClaimsFiled}}{\text{YearsWithoutClaims} + 1}$$

- Policy Duration (Years):

$$\text{PolicyDurationYears} = \frac{\text{PreviousPolicyDurationMonths}}{12}$$

Data augmentation using Gaussian noise was applied during training to improve robustness:

$$X_{augmented} = X + \mathcal{N}(0, \sigma^2)$$

4 Prediction Model

A Random Forest classifier was selected due to:

- Strong performance on tabular data
- Robust Macro F1 performance
- Low inference latency
- Small model footprint
- Compatibility with CPU-only execution

The model predicts:

$$\hat{y} = \arg \max_{c \in \{0, \dots, 9\}} P(c \mid x)$$

where $P(c \mid x)$ is the class probability estimated by the ensemble.

5 Business Optimization Layer

Although the competition focuses on classification, the architecture allows extension toward economic optimization.

Expected gain formulation:

$$\text{ExpectedGain}_i = P_i \times \text{Margin}_i$$

Future extension:

$$\text{Score}_i = P_i \times \text{Margin}_i + \alpha \times \text{RetentionScore}_i$$

This enables long-term value maximization.

6 Explainability

Explainability is achieved using SHAP values:

$$\phi_j = \text{Contribution of feature } j$$

Feature importance and local explanations are provided through a web interface.

7 Deployment Architecture

- Lightweight RandomForest model deployed locally
- API layer for inference
- Cloud optimization service for gain maximization
- Fully offline inference (no network dependency)

8 Compliance with Constraints

- Model size < 50 MB
- Inference latency optimized for single-core CPU
- No internet dependency
- Macro F1 optimized with balanced class handling

9 Conclusion

The proposed system balances performance, efficiency, explainability, and business value while respecting strict computational constraints.